

PROJET ECOTRACK

VEILLE TECHNOLOGIQUE

Master 1 — Filière Développement Logiciel (EADL, RNCP 38822)

I. MÉTHODOLOGIE DE VEILLE TECHNOLOGIQUE

✓ Critères RNCP Ce1.2.1 / A1.2 : Veille structurée et documentée

La veille technologique d'ECOTRACK est conduite de manière structurée et continue afin de fonder chaque choix de stack sur des données objectives et actualisées. Elle repose en priorité sur des sources anglophones de référence, qui concentrent l'essentiel de l'information primaire et la plus récente : documentations officielles des projets (React, FastAPI, PostgreSQL/PostGIS, TimescaleDB, Redis, Mosquitto), benchmarks indépendants (TechEmpower Web Framework Benchmarks), rapports annuels (Stack Overflow Developer Survey, State of JS, ThoughtWorks Technology Radar) et dépôts GitHub des projets évalués (étoiles, fréquence des commits, issues ouvertes, cadence des releases). La maîtrise de l'anglais technique est ici une compétence transversale décisive, l'information francophone étant souvent partielle ou décalée dans le temps.

Les critères de sélection des technologies sont définis a priori et appliqués uniformément : maturité et pérennité, performance mesurée, adéquation aux contraintes du projet (géospatial, séries temporelles, temps réel IoT), taille et vitalité de la communauté, qualité de la documentation, coût total et licence (open source privilégié), et facilité d'intégration dans une architecture microservices conteneurisée. La veille s'organise selon trois rythmes : un scan quotidien de 15 à 30 minutes (agrégateurs RSS, newsletters, GitHub Trending), une analyse hebdomadaire approfondie de 2 heures (lecture de benchmarks et articles de fond), et une synthèse mensuelle partagée en équipe lors d'une Tech Watch. Les outils utilisés sont Feedly (agrégation de plus de 50 sources), Google Alerts sur les mots-clés stratégiques, les comptes d'experts sur X/Twitter et Reddit (r/programming, r/PostgreSQL), et Notion pour la capitalisation structurée des découvertes.

A. Sources Consultées

Sources Techniques :

- GitHub Trending : repos populaires et émergents
- Stack Overflow : questions/réponses trending
- Medium / Dev.to (anglophone) : retours d'expérience et articles techniques récents

- Documentation officielle (anglophone) : React, FastAPI, PostgreSQL, TimescaleDB, MQTT

Conférences et Événements :

- Conférences tech (anglophones) : React Summit, PyCon, FOSDEM, PGConf
- Webinars : Cloud providers (AWS, Azure, GCP)
- Meetups locaux : Paris JS, Python Paris, PostgreSQL France

Veille Réglementaire :

- ANSSI : recommandations sécurité applicables à la plateforme
- CNIL : guides RGPD
- OWASP : Top 10 vulnérabilités web (sources anglophones de référence)

B. Outils Utilisés

- Feedly : agrégateur RSS (50+ sources)
- Google Alerts : mots-clés stratégiques
- Twitter/X : experts tech à suivre
- Reddit (anglophone) : r/programming, r/reactjs, r/PostgreSQL, r/IOT
- Notion/Obsidian : prise de notes structurée

C. Fréquence et Organisation

- Veille quotidienne : 15-30 min (scan rapide)
- Veille hebdomadaire : 2h (analyse approfondie)
- Revue mensuelle : synthèse tendances majeures
- Partage équipe : présentation trimestrielle

II. COMPARAISON DES TECHNOLOGIES

✓ Critères RNCP Ce1.2.2 / A1.2 : Analyse comparative objective

Technologies évaluées (filière Développement)

A. Frameworks Frontend : React vs Vue.js vs Angular

Évaluation sur 5 critères notés de 0 à 10 (total /50). Notes appuyées sur les benchmarks publics (bundle size, Lighthouse) et les rapports State of JS / Stack Overflow Survey.

Critère 1 — Performance (rendu, bundle, optimisations) : React 9 | Vue 9 | Angular 7

Critère 2 — Courbe d'apprentissage / documentation : React 8 | Vue 9 | Angular 6

Critère 3 — Communauté (GitHub, NPM, emploi) : React 10 | Vue 8 | Angular 8

Critère 4 — Écosystème (librairies, tooling, IDE) : React 10 | Vue 8 | Angular 9

Critère 5 — Maintenance (cadence, breaking changes) : React 9 | Vue 8 | Angular 9

TOTAL : React 46/50 | Vue 42/50 | Angular 39/50

CHOIX RECOMMANDÉ : React 18 + TypeScript. React domine sur la communauté et l'écosystème (viviers de développeurs et de librairies les plus larges du marché), facilitant le recrutement et la maintenance long terme. Combiné à TypeScript, il offre un typage strict réduisant les bugs runtime, et s'intègre nativement avec Leaflet (cartographie) et Socket.io (temps réel) requis par ECOTRACK.

B. Frameworks Backend : Node.js/Express vs FastAPI vs Spring Boot

Évaluation sur 5 critères notés de 0 à 10 (total /50). Notes appuyées sur les benchmarks TechEmpower et la documentation officielle des frameworks.

Critère 1 — Performance (req/s, latence) : Node/Express 8 | FastAPI 9 | Spring Boot 8

Critère 2 — Scalabilité (microservices, conteneurs) : Node/Express 9 | FastAPI 9 | Spring Boot 9

Critère 3 — Typage / robustesse : Node/Express 6 | FastAPI 9 | Spring Boot 10

Critère 4 — Async / temps réel (WebSocket, MQTT) : Node/Express 10
| FastAPI 8 | Spring Boot 7

Critère 5 — Rapidité de développement / déploiement : Node/Express 9
| FastAPI 9 | Spring Boot 6

TOTAL : Node/Express 42/50 | FastAPI 44/50 | Spring Boot 40/50

CHOIX RECOMMANDÉ : approche hybride FastAPI (Python) + Node.js. FastAPI est retenu pour l'API métier et le moteur d'optimisation TSP : typage Pydantic, documentation OpenAPI automatique et écosystème Python scientifique (NetworkX, OR-Tools) idéal pour l'algorithme du voyageur de commerce. Node.js est retenu pour les services temps réel (ingestion MQTT, notifications, WebSocket Socket.io) où sa boucle d'événements non bloquante excelle face aux ~500 000 mesures/jour. Spring Boot, bien que très robuste, est écarté pour sa lourdeur de mise en œuvre et son éloignement de la stack Python/JS de l'équipe.

C. Bases de Données : PostgreSQL vs MongoDB vs TimescaleDB

Évaluation sur 5 critères notés de 0 à 10 (total /50), focalisée sur les cas d'usage ECOTRACK (géospatial des conteneurs et tournées, séries temporelles des mesures de niveau).

Critère 1 — Performance (charge d'écriture IoT) : PostgreSQL 8 | MongoDB 8 | TimescaleDB 10

Critère 2 — Scalabilité / volumétrie : PostgreSQL 8 | MongoDB 9 | TimescaleDB 9

Critère 3 — Requêtes complexes (jointures, SQL géospatial) : PostgreSQL 10 | MongoDB 6 | TimescaleDB 9

Critère 4 — Extensions (géospatial, séries temporelles) : PostgreSQL 10 | MongoDB 7 | TimescaleDB 10

Critère 5 — Coût / licence (open source) : PostgreSQL 10 | MongoDB 7 | TimescaleDB 9

TOTAL : PostgreSQL 46/50 | MongoDB 37/50 | TimescaleDB 47/50

CHOIX RECOMMANDÉ : PostgreSQL 15 avec l'extension PostGIS comme base de données principale (relationnel, géospatial pour conteneurs et tournées, requêtes complexes), complété par TimescaleDB (extension PostgreSQL) pour les séries temporelles des ~500 000 mesures/jour. Cette combinaison conserve un moteur unique et un dialecte SQL homogène — TimescaleDB étant une extension de

PostgreSQL, l'intégration PostGIS + TimescaleDB se fait sans rupture technologique. MongoDB, malgré sa souplesse, est écarté : la modélisation géospatiale et les requêtes analytiques complexes y sont moins matures que sous PostgreSQL/PostGIS.

III. TENDANCES ET INNOVATIONS 2025

A. Tendances Globales (Toutes Filières)

1. IA Générative et LLMs

- Impact : Assistants intelligents pour chauffeurs, génération rapports automatisés
- Technologies : GPT-4, Claude, LLaMA, fine-tuning
- Application ECOTRACK : Chatbot support chauffeurs, analyse prédictive textuelle

2. Edge Computing et IoT

- Impact : Traitement local sur capteurs, réduction latence
- Technologies : Edge AI, TensorFlow Lite, MQTT 5.0
- Application ECOTRACK : Pré-traitement données capteurs, alertes offline

3. Zero Trust Security

- Impact : Sécurisation granulaire des accès, conformité renforcée
- Technologies : Authentification continue, micro-segmentation
- Application ECOTRACK : Accès chauffeurs/gestionnaires par rôles stricts

B. Tendances Spécifiques par Filière

Tendances 2025 pertinentes pour ECOTRACK (Développement)

1. React Server Components (RSC) : Rendu hybride client/serveur
→ Impact ECOTRACK : Performance améliorée, SEO optimisé
2. WebAssembly (WASM) : Code natif dans navigateur
→ Impact : Algorithmes lourds (TSP) exécutés côté client
3. TypeScript 5+ : Typage encore plus strict
→ Impact : Réduction bugs runtime, maintenabilité
4. Micro-Frontends : Architecture modulaire frontend
→ Impact : Équipes autonomes, déploiement indépendant modules

IV. CHOIX TECHNOLOGIQUES FINAUX POUR ECOTRACK



Critères RNCP Ce1.2.3 / A1.2 : Choix justifiés et documentés

A. Synthèse des Choix par Composant

—
COMPOSANT : Frontend

TECHNOLOGIE RETENUE : React 18 + TypeScript

SCORE COMPARATIF : 46/50

JUSTIFICATION :

1. Plus large communauté et écosystème du marché (recrutement et maintenance facilités).
2. Typage strict TypeScript réduisant les bugs runtime sur une application métier complexe.
3. Intégration native avec Leaflet (cartographie) et Socket.io (temps réel) requis par ECOTRACK.

RISQUES : verbosité et configuration initiale ; mitigés par l'usage de Vite et d'une base de composants standardisée.

—
COMPOSANT : Backend

TECHNOLOGIE RETENUE : FastAPI (API métier + optimisation TSP) + Node.js (services temps réel)

SCORE COMPARATIF : 44/50 (FastAPI) / 42/50 (Node.js)

JUSTIFICATION :

1. FastAPI : typage Pydantic, doc OpenAPI auto et écosystème Python (OR-Tools, NetworkX) pour le TSP.
2. Node.js : boucle d'événements non bloquante optimale pour MQTT, WebSocket et notifications temps réel.

3. Découpage en 5 microservices (IoT, API, Optim, Auth, Notif) permettant un scaling indépendant.

RISQUES : surcoût d'exploitation de deux runtimes ; mitigé par la conteneurisation Docker et un découpage clair des responsabilités.

—
COMPOSANT : Base de données

TECHNOLOGIE RETENUE : PostgreSQL 15 + PostGIS + TimescaleDB

SCORE COMPARATIF : 47/50

JUSTIFICATION :

1. PostGIS couvre le géospatial (positions des conteneurs, calcul de tournées) en SQL natif.

2. TimescaleDB absorbe les ~500 000 mesures/jour avec compression et agrégats continus performants.

3. Moteur unique PostgreSQL : dialecte SQL homogène, une seule compétence d'exploitation, open source.

RISQUES : volumétrie des séries temporelles ; mitigée par les politiques de rétention et la compression TimescaleDB.

B. Stack Technique Complète Recommandée

Frontend : React 18 + TypeScript, Vite, Leaflet + OpenStreetMap, Recharts, Socket.io client

Backend : FastAPI (Python) pour l'API métier et l'optimisation TSP, Node.js pour les services temps réel ; architecture microservices (IoT, API, Optim, Auth, Notif)

Base de données : PostgreSQL 15 + PostGIS (géospatial) + TimescaleDB (séries temporelles) ; Redis (cache et files)

Infrastructure : Docker (conteneurisation), Nginx (reverse proxy), MQTT Mosquitto (broker IoT), WebSocket (Socket.io)

Monitoring : Prometheus + Grafana (métriques), alerting sur CVE et breaking changes (Dependabot / Renovate)

Sécurité : authentification JWT, 2FA TOTP, contrôle d'accès par rôles (RBAC), chiffrement TLS

CI/CD : GitHub Actions (build, tests, image Docker), déploiement automatisé, méthodologie Scrum (8 sprints de 2 semaines)

V. PLAN DE VEILLE CONTINUE

A. Pendant le Projet (16 semaines)

- Veille hebdomadaire : 1h par développeur (technologies utilisées)
- Sprint Review : partage découvertes et bonnes pratiques
- Tech Watch : présentation mensuelle tendances équipe
- Alertes critiques : CVE sécurité, breaking changes librairies

B. Post-Déploiement (Maintenance)

- Monitoring versions : Dependabot + Renovate Bot
- Veille sécurité : ANSSI, OWASP, CVE databases
- Roadmaps officielles (anglophones) : React, FastAPI, PostgreSQL, TimescaleDB
- Participation communauté : Conférences, meetups, contributions open-source

C. Ressources Recommandées

- Newsletters (anglophones) : JavaScript Weekly, Python Weekly, Postgres Weekly, Node Weekly
- Podcasts (anglophones) : Syntax.fm, Talk Python To Me, Software Engineering Daily
- YouTube (anglophones) : Fireship, Hussein Nasser, ArjanCodes
- Formation continue : Udemy, Pluralsight, Cloud certifications

FIN DE LA VEILLE TECHNOLOGIQUE
